

N+1 Query Problem & Solutions

The core problem is simple:

*You execute **1 query to load the parent records**, then Hibernate executes **N additional queries** to load a related collection/entity for those N parents.*

Hibernate explicitly identifies `SELECT`-based association fetching as a common source of N+1 behavior. ([Hibernate Documentation](#))

1. Start With a Simple Example

Suppose:

```
Customer
|
├─ Payment 1
├─ Payment 2
└─ Payment 3
```

Entities:

```
@Entity
public class Customer {

    @Id
    private Long id;

    @OneToMany(mappedBy = "customer",
               fetch = FetchType.LAZY)
    private List<Payment> payments;
}
```

```
@Entity
public class Payment {

    @Id
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "customer_id")
    private Customer customer;
}
```

Now:

```
List<Customer> customers =
    customerRepository.findAll();
```

Hibernate executes approximately:

```
SELECT *  
FROM customer;
```

Suppose that returns:

```
Customer 1  
Customer 2  
Customer 3  
...  
Customer 100
```

Then your code does:

```
for (Customer customer : customers) {  
    customer.getPayments().size();  
}
```

Because `payments` is lazy, Hibernate may now issue:

```
SELECT *  
FROM payment  
WHERE customer_id = 1;  
  
SELECT *  
FROM payment  
WHERE customer_id = 2;  
  
SELECT *  
FROM payment  
WHERE customer_id = 3;  
  
...
```

So:

```
1 query  
+  
100 additional queries  
=  
101 queries
```

That's the **N+1 problem**.

2. Why Is It Bad?

The obvious issue is more SQL.

But the bigger problem is:

```
More DB round trips  
↓  
More network overhead  
↓  
More DB work  
↓  
Higher latency  
↓
```

Poor scalability

For:

10 customers

you might barely notice it.

For:

10,000 customers

you can have:

10,001 queries

That can become a serious production performance problem.

3. Why Does Lazy Loading Cause It?

LAZY isn't inherently bad.

In fact, lazy loading is often preferable because it avoids loading relationships that you don't need.

The problem is the combination:

```
LAZY relationship
+
loop over many parent entities
+
access relationship inside loop
```

For example:

```
for (Customer customer : customers) {
    customer.getPayments();
}
```

You are effectively saying:

"Load the association separately for each customer."

That's where N+1 can emerge.

Hibernate's documentation recommends using **LAZY** associations and fetching them eagerly at the query/use-case level rather than relying on globally eager relationships. ([Hibernate Documentation](#))

4. Best-Known Solution: **JOIN FETCH**

Instead of:

```
@Query("""
    SELECT c
    FROM Customer c
""")
List<Customer> findCustomers();
```

use:

```
@Query("""
    SELECT DISTINCT c
    FROM Customer c
    JOIN FETCH c.payments
""")
List<Customer> findCustomersWithPayments();
```

The important keyword is:

JOIN FETCH

A fetch join tells Hibernate to retrieve the association as part of the SQL join rather than triggering separate selects. Hibernate specifically documents `join fetch` as a key solution to N+1 selects. ([Hibernate Documentation](#))

5. What SQL Does This Produce?

Conceptually:

```
SELECT
    c.id,
    c.name,
    p.id,
    p.amount,
    p.status
FROM customer c
JOIN payment p
ON p.customer_id = c.id;
```

Instead of:

1 + N queries

you can get:

1 query

So:

```
Without JOIN FETCH

Customer query
+
Payment query per customer
↓
N+1

With JOIN FETCH
```

```
Customer + Payment
  ↓
Single joined query
```

6. Why **DISTINCT** ?

Suppose:

```
Customer 1
  Payment A
  Payment B
  Payment C
```

The SQL join produces multiple rows:

```
Customer 1 | Payment A
Customer 1 | Payment B
Customer 1 | Payment C
```

But at the Java object level, you want:

```
Customer 1
  payments = [A, B, C]
```

So you'll commonly see:

```
SELECT DISTINCT c
FROM Customer c
JOIN FETCH c.payments
```

The **DISTINCT** is primarily useful at the object/query result level to avoid duplicate root entities in the result.

7. **JOIN FETCH** vs Normal **JOIN**

This distinction is important.

Normal JOIN

```
SELECT c
FROM Customer c
JOIN c.payments p
```

The join can be used for filtering/query logic, but it doesn't necessarily tell Hibernate:

*"Populate **c.payments** as part of this query."*

JOIN FETCH

```
SELECT c
FROM Customer c
JOIN FETCH c.payments
```

means:

Join the association and fetch it into the entity graph.

So:

```
JOIN
→ query relationship

JOIN FETCH
→ query relationship + initialize/fetch association
```

8. JOIN FETCH vs LEFT JOIN FETCH

Suppose some customers have no payments.

JOIN FETCH

```
JOIN FETCH c.payments
```

behaves like an inner join.

Customers without matching payments may be excluded.

LEFT JOIN FETCH

```
LEFT JOIN FETCH c.payments
```

includes:

```
Customers with payments
+
Customers without payments
```

Hibernate documents this distinction explicitly. ([Hibernate Documentation](#))

So:

```
Need only parents having children
→ JOIN FETCH

Need all parents, whether child exists or not
→ LEFT JOIN FETCH
```

9. Spring Data JPA Example

Repository:

```
public interface CustomerRepository
    extends JpaRepository<Customer, Long> {

    @Query("""
        SELECT DISTINCT c
        FROM Customer c
        LEFT JOIN FETCH c.payments
        """)
    List<Customer> findAllWithPayments();
}
```

Now:

```
List<Customer> customers =
    repository.findAllWithPayments();

for (Customer customer : customers) {
    customer.getPayments().size();
}
```

doesn't need one query per customer to initialize the collection.

10. Solution 2 — @EntityGraph

Spring Data JPA also supports **Entity Graphs** for defining which associations should be fetched for a particular repository operation. ([Home](#))

Example:

```
@EntityGraph(attributePaths = {"payments"})
List<Customer> findAll();
```

Conceptually:

```
findAll()
+
fetch "payments"
```

This can be cleaner than writing JPQL for straightforward fetching requirements.

Spring Data JPA's `@EntityGraph` supports both named graphs and ad-hoc attribute paths. ([Home](#))

11. @EntityGraph Example

```
public interface CustomerRepository
    extends JpaRepository<Customer, Long> {

    @EntityGraph(attributePaths = "payments")
    List<Customer> findAll();
}
```

Now the repository method communicates:

"For this query, I need customers together with their payments."

This is particularly useful because you don't have to make the mapping itself permanently eager.

12. Why Not Just Use **EAGER** ?

A common incorrect answer:

"N+1 happens because it's LAZY, so change everything to EAGER."

Don't do that.

Suppose you have:

```
Customer
├─ Payments
├─ Address
├─ Preferences
├─ AuditRecords
└─ Notifications
```

Making everything eager could cause huge object graphs and unnecessary loading.

Worse, Hibernate can still produce secondary selects in some JPQL/entity-query scenarios even for eager associations; Hibernate's documentation explicitly warns that omitted eager associations in entity queries can lead to secondary selects and N+1 behavior. ([Hibernate Documentation](#))

Better:

```
Keep associations appropriately LAZY
↓
Fetch required relationships
↓
at query/use-case level
```

13. Solution 3 — Batch Fetching

Sometimes one **JOIN FETCH** isn't the best fit.

Hibernate supports batch fetching.

Instead of:

```
Customer 1 → query payments
Customer 2 → query payments
Customer 3 → query payments
...
```

Hibernate can batch IDs:

```
SELECT *
FROM payment
```

```
WHERE customer_id IN (1, 2, 3, 4, 5);
```

Then another batch:

```
WHERE customer_id IN (6, 7, 8, 9, 10);
```

Hibernate documents batch fetching as an alternative fetching strategy. ([Hibernate Documentation](#))

14. @BatchSize

For example:

```
@BatchSize(size = 20)
@OneToMany(mappedBy = "customer")
private List<Payment> payments;
```

The exact optimal batch size depends on the workload and database.

Conceptually:

```
N individual queries
↓
N / batchSize queries
```

For example:

```
100 customers
batch size = 20

≈ 5 association queries
```

instead of 100.

Hibernate documents `@BatchSize` as a mechanism for batching lazy association/entity initialization. ([Hibernate Documentation](#))

15. Batch Fetching vs JOIN FETCH

JOIN FETCH	Batch fetching
Can fetch in same SQL query	Uses additional SELECTs
Often 1 query	Usually multiple batched queries
Great when data is definitely needed	Useful when lazy loading remains desirable
Can create large joins	Less join explosion
Can be awkward with pagination	More flexible in some scenarios

A good rule:

Need parent + relationship now?

↓

JOIN FETCH / EntityGraph

Need lazy relationship but want to reduce N+1?

↓

Batch fetching

16. Solution 4 — DTO Projection

Sometimes you don't actually need full entities.

Suppose the API only needs:

```
customerId
customerName
paymentStatus
paymentAmount
```

Don't necessarily load a huge entity graph.

Use a projection/DTO query:

```
@Query("""
    SELECT new com.example.dto.CustomerPaymentDto(
        c.id,
        c.name,
        p.status,
        p.amount
    )
    FROM Customer c
    JOIN c.payments p
""")
List<CustomerPaymentDto> findCustomerPayments();
```

Now Hibernate retrieves only the data required for the use case.

Hibernate's own documentation notes that DTO projection can often be preferable to batch fetching or unnecessary entity loading, depending on the use case. ([Hibernate Documentation](#))

17. Solution 5 — Subselect Fetching

Hibernate also supports **subselect fetching**.

Conceptually:

```
First query:
SELECT customers ...

Then:
SELECT payments
FROM payment
WHERE customer_id IN (
    IDs obtained from the original customer query
)
```

This can reduce N+1 in certain access patterns.

However, for most interviews, you should prioritize:

1. JOIN FETCH
2. EntityGraph
3. Batch fetching
4. DTO projection

and then know that Hibernate also has subselect fetching as an alternative. ([Hibernate Documentation](#))

18. The Pagination Trap

This is very important.

Suppose:

```
Page<Customer> findAll(Pageable pageable);
```

and you want:

```
JOIN FETCH c.payments
```

This becomes tricky when the fetched association is a **collection**.

Why?

Suppose:

```
10 customers per page
```

but each customer has:

```
100 payments
```

The SQL join can produce:

```
1000 rows
```

even though you logically requested:

```
10 customers
```

The database's `LIMIT/OFFSET` applies to the joined result, not your conceptual root-entity list, which can make correct pagination difficult.

Hibernate specifically advises avoiding fetch joins in limited/paged queries. ([Hibernate Documentation](#))

19. Better Pagination Strategy

For collection relationships, a common strategy is:

Query 1

Get the page of customer IDs:

```
SELECT id
FROM customer
ORDER BY id
LIMIT 20 OFFSET 40;
```

Query 2

Fetch those customers and their payments:

```
SELECT ...
FROM customer c
LEFT JOIN payment p
  ON ...
WHERE c.id IN (...20 IDs...);
```

Conceptually:

```
Page IDs
↓
Fetch entities
+
Fetch relationships
```

This avoids trying to paginate directly over a collection fetch join.

20. The Cartesian Product Problem

Suppose you do:

```
SELECT DISTINCT c
FROM Customer c
JOIN FETCH c.payments
JOIN FETCH c.addresses
```

and both `payments` and `addresses` are collections.

Imagine:

```
Customer 1
  100 payments
  20 addresses
```

The database join can produce roughly:

```
100 × 20 = 2,000 rows
```

for that one customer.

Hibernate explicitly warns that fetching multiple to-many associations in parallel can produce a Cartesian product and poor performance. ([Hibernate Documentation](#))

Therefore:

Don't blindly add `JOIN FETCH` to every relationship.

21. `JOIN FETCH` Can Be Nested

Fetching a to-one association and then another nested association can be reasonable.

Example:

```
SELECT o
FROM Order o
JOIN FETCH o.customer c
JOIN FETCH c.address
```

Hibernate documents that multiple to-one fetches and a nested series of fetch joins can be safe, while parallel collection fetches are the major danger. ([Hibernate Documentation](#))

22. A Very Practical Example

Suppose your API is:

```
GET /customers
```

and response requires:

```
[
  {
    "id": 1,
    "name": "Avnish",
    "payments": [
      {
        "id": 101,
        "amount": 500
      },
      {
        "id": 102,
        "amount": 700
      }
    ]
  }
]
```

Bad approach

```
List<Customer> customers =
    repository.findAll();

for (Customer customer : customers) {
    customer.getPayments();
}
```

Potential result:

```
1 customer query
```

+
N payment queries

Better

```
@Query("""
    SELECT DISTINCT c
    FROM Customer c
    LEFT JOIN FETCH c.payments
    """)
List<Customer> findCustomersWithPayments();
```

Then:

1 query

for that particular fetch plan, subject to the SQL/result shape and provider behavior.

23. Important Misconception

“N+1 always means exactly N+1 SQL queries.”

Not necessarily.

The classic form is:

1 + N

but additional lazy associations or nested traversals can produce more complicated query counts.

Think of N+1 as:

A scaling pattern where accessing a relationship causes additional queries proportional to the number of parent records.

24. How to Detect N+1

Don't rely only on reading the code.

Look at SQL.

Enable Hibernate SQL logging in development, for example:

```
spring.jpa.show-sql=true
```

or configure appropriate Hibernate logging.

In production, you can use:

- application SQL logs
- APM

- database monitoring
- Hibernate statistics
- query-count tests

The key thing to inspect is:

```
One parent query
+
Repeated similar child queries
=
Likely N+1
```

Example:

```
SELECT ... FROM customer

SELECT ... FROM payment WHERE customer_id=1
SELECT ... FROM payment WHERE customer_id=2
SELECT ... FROM payment WHERE customer_id=3
SELECT ... FROM payment WHERE customer_id=4
...
```

That pattern is the giveaway.

25. Choosing the Solution

A useful decision table:

Situation	Good starting approach
Need parent + one relationship	<code>JOIN FETCH</code>
Simple repository fetch plan	<code>@EntityGraph</code>
Large lazy collection accessed across many parents	Batch fetching
API needs only selected fields	DTO projection
Paginated parent + collection	Two-step query / tailored query
Multiple large to-many relationships	Avoid one giant fetch join
Relationship not needed	Keep it lazy / don't fetch it

26. What Not to Do

Don't do this blindly

```
@OneToMany(fetch = FetchType.EAGER)
```

everywhere.

Don't do this blindly

```
JOIN FETCH
payments
JOIN FETCH
transactions
JOIN FETCH
history
JOIN FETCH
...
```

You can create enormous SQL result sets.

Don't do this

```
findAll()
```

then access several lazy collections inside loops without understanding query behavior.

27. Interview-Ready Answer

If asked:

“What is the N+1 query problem and how do you solve it?”

Say:

N+1 occurs when Hibernate executes one query to load a collection of parent entities and then executes an additional query for each parent when a lazy association is accessed. For example, loading 100 customers with `findAll()` and then accessing each customer's payments can result in 101 queries. I would first identify the required fetch plan rather than changing everything to EAGER. For relationships that are required for the use case, I can use JPQL `JOIN FETCH` or Spring Data's `@EntityGraph`. For cases where lazy loading is still appropriate, batch fetching can reduce the number of secondary queries. For read-only APIs where I only need selected fields, a DTO projection can be better. I would also be careful with collection fetch joins and pagination because they can produce large Cartesian result sets and problematic pagination behavior.

That's a **strong 3-year-level answer**.

28. BPS-Specific Questions — Inspect Your Codebase

Now inspect your actual BPS code rather than assuming the implementation.

Search for:

```
@OneToMany
@ManyToOne
@OneToOne
@Query
JOIN FETCH
@EntityGraph
```

```
@BatchSize  
findAll()
```

Then trace one real API.

Question 1

Does BPS load something like:

```
Payment  
↓  
Payment History  
↓  
Vendor / Customer / Policy
```

and subsequently access those relationships inside loops?

Question 2

Do any repository queries contain:

```
JOIN FETCH
```

or:

```
@EntityGraph
```

?

Question 3

Do you see:

```
FetchType.LAZY
```

on BPS entities?

What happens when the associated collection is accessed?

Question 4

Does BPS use pagination for any large payment/history query?

If yes, check carefully whether a collection is being fetched alongside the paginated root entity.

Question 5 — Interview scenario

Suppose BPS retrieves 5,000 payment records and then accesses payment history for each record. How would you determine whether an N+1 problem exists, and how would you redesign the query?

A good investigation path is:

```
Check SQL logs  
↓  
Count repeated SELECTs  
↓
```

Identify association causing them

↓

Check required response data

↓

Choose:

JOIN FETCH

OR

@EntityGraph

OR

batch fetching

OR

DTO projection

OR

two-step paginated query

That is the practical Hibernate skill interviewers are usually looking for—not simply memorizing what N+1 stands for.